Solution: Remove Nth Node From End of List

Problem Statement

Given a linked list, remove the last `nth` node from the end of the list and return the head of the modified list.

Example 1:

- **Input**: list = 1 -> 2 -> 3 -> 4 -> 5, n = 2
- **Expected Output**: 1 -> 2 -> 3 -> 5
- **Justification**: The 2nd node from the end is "4", so after removing it, the list becomes [1,2,3,5].

Example 2:

- **Input**: list = 10 -> 20 -> 30 -> 40, n = 4
- **Expected Output**: 20 -> 30 -> 40
- **Justification**: The 4th node from the end is "10", so after removing it, the list becomes [20,30,40].

Example 3:

- **Input**: list = 7 -> 14 -> 21 -> 28 -> 35, n = 3
- **Expected Output**: 7 -> 14 -> 28 -> 35
- **Justification**: The 3rd node from the end is "21", so after removing it, the list becomes [7,14,28,35].

Solution

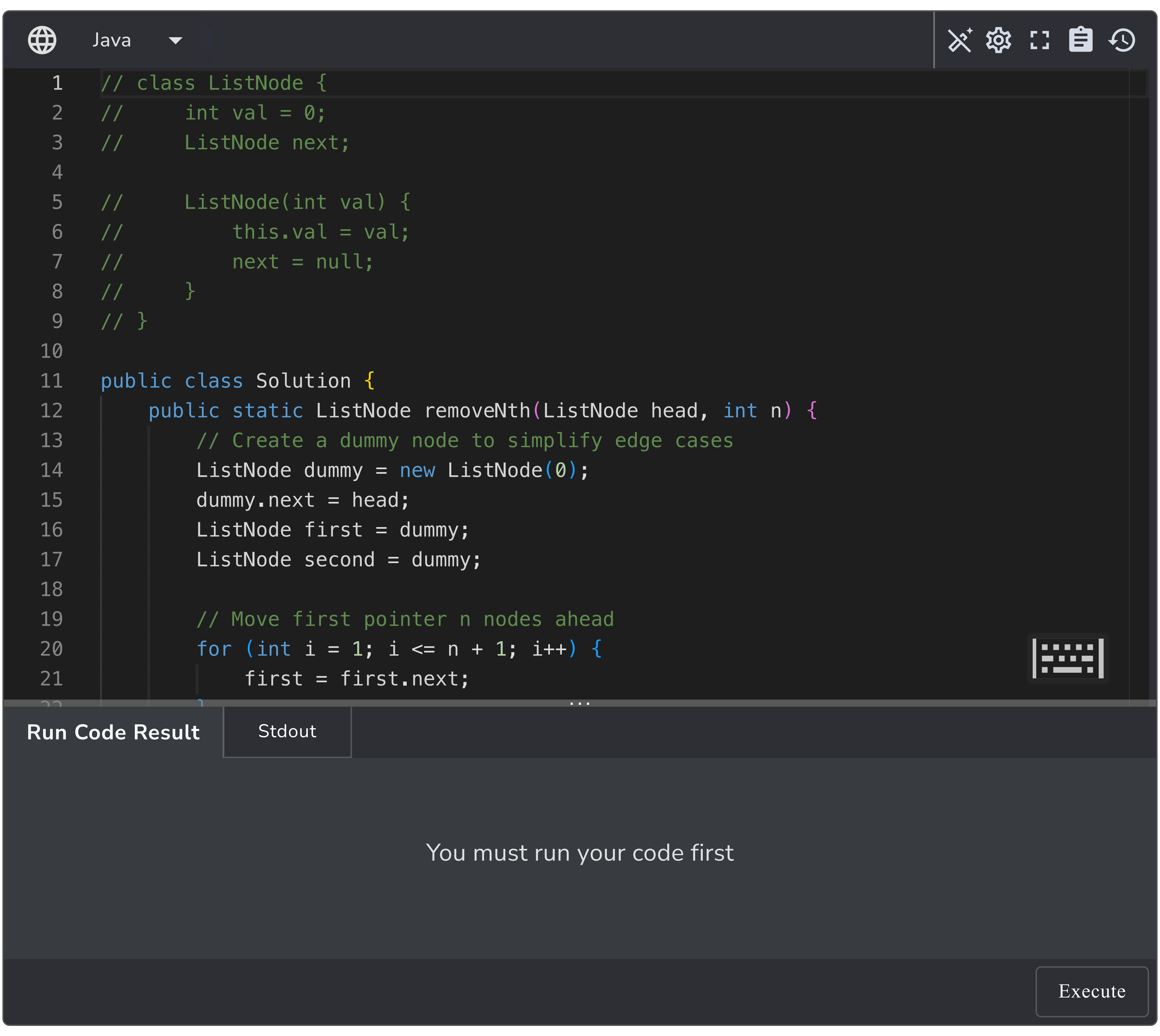
- **Two-Pass Approach**:
  - Begin by calculating the length of the linked list. This can be done by traversing the list from the head to the end.
  - Once the length is determined, calculate which node to remove by subtracting `n` from the length.
  - Traverse the list again and remove the node at the calculated position.
- **One-Pass Approach using Two Pointers**:
  - Use two pointers, `first` and `second`, and place them at the start of the list.
  - Move the `first` pointer `n` nodes ahead in the list.
  - Now, move both `first` and `second` pointers one step at a time until the `first` pointer reaches the end of the list. The `second` pointer will now be `n` nodes from the end.
  - Remove the node next to the `second` pointer.
- **Advantage of One-Pass Approach**:
  - The one-pass approach is more efficient as it traverses the list only once, whereas the two-pass approach requires two traversals.
- **Edge Cases**:
  - If `n` is equal to the length of the list, remove the head of the list.

Algorithm Walkthrough

Using the input list = [1,2,3,4,5], n = 2:

1. Initialize two pointers, `first` and `second`, at the head of the list.
2. Move the `first` pointer 2 nodes ahead. Now, `first` points to "3" and `second` points to "1".
3. Move both `first` and `second` pointers one step at a time. When `first` reaches "5", `second` will be at "3".
4. The next node to `second` is "4", which is the node to be removed.
5. Remove "4" by updating the next pointer of "3" to point to "5".
6. The modified list is [1,2,3,5].

Code



```java
// class ListNode {
//     int val = 0;
//     ListNode next;

//     ListNode(int val) {
//         this.val = val;
//         next = null;
//     }
// }

public class Solution {
    public static ListNode removeNth(ListNode head, int n) {
        // Create a dummy node to simplify edge cases
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode first = dummy;
        ListNode second = dummy;

        // Move first pointer n nodes ahead
        for (int i = 1; i <= n + 1; i++) {
            first = first.next;
```

You must run your code first

Complexity Analysis

- **Time Complexity**: O(L) - We traverse the list with two pointers. Here, L is the number of nodes in the list.
- **Space Complexity**: O(1) - We only used constant extra space.

← Previous
Remove Nth Node From End Of List (Medium)

Next →
Find Minimum In Rotated Sorted Array (Medium)

☐ Mark as Completed